

Flow Problems and Integer Programs

1 Introduction

The Minimum Cost Flow and the Maximum $s - t$ Flow problems are continuous (linear) problems, since there is no restriction that imposes that the flow on the arcs must be integer. However, they are special continuous problems, in the sense that they can be used to solve certain integer programs. In these notes, we will see some of these integer programs.

2 The Minimum Cost Flow problem

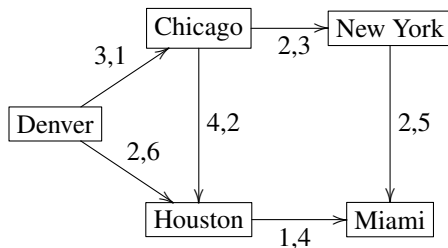
Recall that in the Minimum Cost Flow problem we are given:

- A network $N(V, A)$;
- For each node $v \in V$ a demand/offer d_v ;
- For each arc $a \in A$, a cost c_a and a capacity u_a ;

and we want to find, among all flows satisfying all demands / offers of the nodes and respecting capacities, one of minimum cost.

We repeat below an example taken from the lecture notes of a previous class.

Example. Consider the instance given by the network $N(V, A)$ below,



where, on every edge, the first number denotes the capacity and the second denotes the cost. We moreover have the following demands/offers.

City v	Denver	Chicago	Houston	New York	Miami
d_v	-4	-2	3	1	2

We can find an optimal solution for the Minimum Cost Flow problem above as a solution to the Linear Program below

$$\begin{aligned}
\min \quad & c_{DC} \cdot x_{DC} + c_{DH} \cdot x_{DH} + c_{CH} \cdot x_{CH} + c_{CN} \cdot x_{CN} + c_{HM} \cdot x_{HM} + c_{NM} \cdot x_{NM} \\
& \underbrace{\sum_{a=(u,v) \in A} x_a}_{\text{flow into } v} - \underbrace{\sum_{a=(v,u) \in A} x_a}_{\text{flow out of } v} = d_v \quad \text{for } v \in \{D, C, H, N, M\} \\
& x_a \leq u_a \quad \text{for all } a \in A \\
& x \geq 0
\end{aligned}$$

The equalities are usually called the *flow balance* constraints. The first set of inequalities are called *capacity constraints* while the second set are called *nonnegativity*.

Note that in the Minimum Cost Flow problem, no constraint forces variables to be integer; however, under very general conditions, the problem has an optimal integer solution.

Theorem 1. *If in a feasible instance of the Minimum Cost Flow vectors d and u are integer, then there is an integer optimal solution, which is also a basic feasible solution.*

The theorem above has a number of important implications. First, it implies that in all the transportation, logistic, etc. problems seen as applications of Minimum Cost Flow, we will obtain an optimal solution that is integer, without explicitly imposing the integrality constraint on the variables. This is very useful if in our original application the flow was, indeed, an integer quantity (such as trucks of a given goods to be sent between a warehouse and a store).

We will see next how we can use the integrality of optimal solutions to Minimum Cost Flow problems to solve other integer problems.

2.1 Shortest paths via min-cost flow

Consider the following Shortest Path Problem with nonnegative costs.

Given: A network $N(V, A)$ with source s and sink t and costs $c : A \rightarrow \mathbb{N}$;

Find: A path from s to t in N with the smallest total cost (“shortest path”), where the cost of a path is the sum of the costs of the arcs that belong to it.

See Figure 1, left, for an instance of the Shortest Path Problem with nonnegative costs.

An optimal solution to the Shortest Path Problem with nonnegative costs can be obtained from the optimal solution to the following Minimum Cost Flow problem on $N(V, A)$:

- Assign to each arc a of N capacity $u_a = 1$;
- For each arc a of N , define a cost c_a as given in input to the shortest path problem (so $c : A \rightarrow \mathbb{N}$);
- Let $d_s = -1$, $d_t = 1$, and $d_v = 0$ for all other $v \in V$.

Indeed, let x^* be the optimum Minimum Cost Flow from the instance above. Since capacities are equal to 1, from Theorem 1 we know that we can assume $x_a^* \in \{0, 1\}$ for each arc $a \in A$. Define P to be the set of arcs $a \in A$ with $x_a^* = 1$. The following can then be easily shown true.

Theorem 2. *The path P constructed as above is a shortest $s - t$ path.*

See Figure 1 for an example.

Let us consider the following generalization of the Shortest Path Problem seen above, where arcs may also have negative costs.

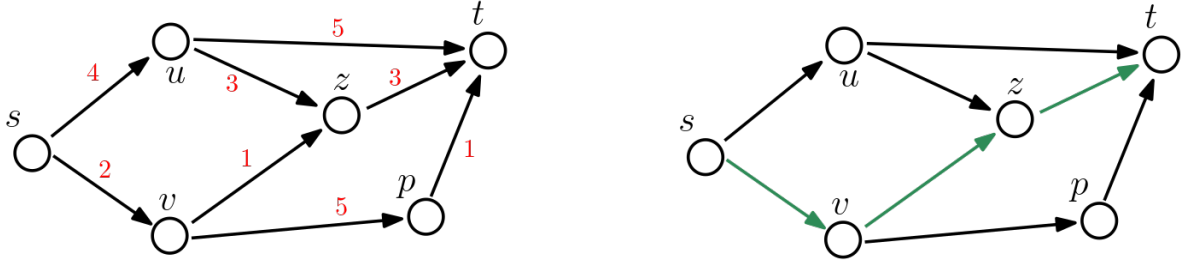


Figure 1: On the left: an instance of the Shortest Path problem between s and t . Next to each arc, we denote its cost. On the right: Its formulation as a Minimum Cost Flow problem, where the cost of each arc coincides with the cost in the network on the left, each arc has a capacity of 1, and the demand d satisfies $d_s = -1$, $d_t = 1$, $d_i = 0$ for $i \neq s, t$. In green, we report the arcs a with $x_a^* = 1$ for the optimal solution x^* to the Minimum Cost Flow problem; all other arcs a satisfy $x_a^* = 0$. By Theorem 2, the green arcs also give the path of minimum cost (“the shortest path”) between s and t .

Given: A network $N(V, A)$ with source s and sink t and costs $c : A \rightarrow \mathbb{Z}$;

Find: A path from s to t in N with the smallest total cost, where the cost of a path is the sum of the costs of the arcs that belong to it.

Can we still solve it via reduction to Minimum Cost Flow? It turns out that we can, but only under certain conditions. A *cycle* in a network is exactly what we expect it to be: a sequence of nodes and arcs that follow one another in the network, leading back to the original node. More formally, a cycle in a network is a sequence:

$$v_1, a_1, v_2, a_2, \dots, v_{n-1}, a_{n-1}, v_1 \quad (1)$$

such that:

- $v_1, \dots, v_{n-1}$ are distinct nodes of the network;
- $a_{n-1} = (v_{n-1}, v_1)$ and for each $i = 1, \dots, n-2$, $a_i = (v_i, v_{i+1})$ are arcs of the network.

A network is *acyclic* if it has no cycle.

Theorem 3. Consider an instance of the Shortest Path Problem where some of the arcs have negative costs. Suppose that there is no negative cost cycle – that is, there is no cycle as in (1) such that $c(a_1) + c(a_2) + \dots + c(a_{n-1}) < 0$. Then the reduction to Minimum Cost Flow discussed above can again be used to produce a shortest path between s and t .

Interestingly, we do not know of any fast algorithm to solve the Shortest Path Problem¹ for generic costs (that is, costs that produce cycles of negative costs).

2.2 An application in project management

Consider the following project management problem: suppose that we are given a sequence of activities $1, \dots, n$, where each activity i has an *execution time* c_i and a list of *prerequisite* activities that need to be completed before activity i is started. The goal is to find the *project completion time*, that is, the time required to complete all activities, assuming that we can perform in parallel any set of activities that are not prerequisite of one another.

¹This problem is known to be NP-hard.

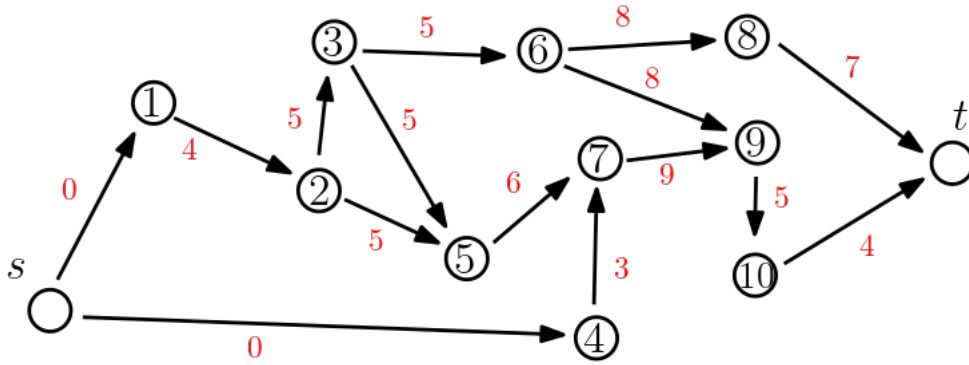


Figure 2: The network representation of the instance of the problem in project management.

An example of this problem is given below.

activity	1	2	3	4	5	6	7	8	9	10
c_i	4	5	5	3	6	8	9	7	5	4
prerequisite	none	1	2	none	2,3	3	4,5	6	6,7	9

We can represent this instance as a Network $N(V, A)$ as follows:

- Nodes are given by activities, plus a source s and a sink t ;
- We have an arc (s, i) of cost 0 for all activities i without a prerequisite;
- We have an arc (i, t) of cost c_i for all activities i that are not prerequisite of other activities;
- We have an (i, j) of cost c_i for all activities i, j such that i is a prerequisite of j .

See Figure 2 for the network corresponding to the example above.

What does a path from s to t in $N(V, A)$ correspond to? It corresponds to a sequence of activities that are prerequisites of one another. For instance, in Figure 2, we have the path $s, 1, 2, 5, 7, 9, 10, t$, and indeed activity 10 can start only once activity 9 has been completed, activity 9 can only start once activity 7 has been completed, etc. The total cost of the path is the sum of the execution times of the activities on the path. From what argued above, it is a lower bound to the project completion time.

Hence, paths with higher costs (“longer paths”) give more accurate bounds to the completion time of the project. For instance, the path $s, 1, 2, 5, 7, 9, 10, t$ gives a lower bound of 33, while the path $s, 4, 7, 9, 10, t$ gives a worse lower bound of 21. It turns out that the longest path from s to t in the network gives exactly the project completion time.

Theorem 4. *Consider an instance of the project management problem, and the corresponding network $N(V, A)$ constructed as above. The path from s to t with largest total cost (“longest path”) corresponds to the project completion time.*

Theorem 4 tells us that we can find the project completion time by finding a longest path from s to t in a network – but how can we find the latter? First observe that this problem is equivalent to finding a shortest path in a network where each edge cost c_a is replaced by $-c_a$.

We know from Section 2.1 that a shortest path in a network can be found via Minimum Cost Flow, as long as the network has no negative cost cycle. Notice that a network arising from a project management instance has no cycle at

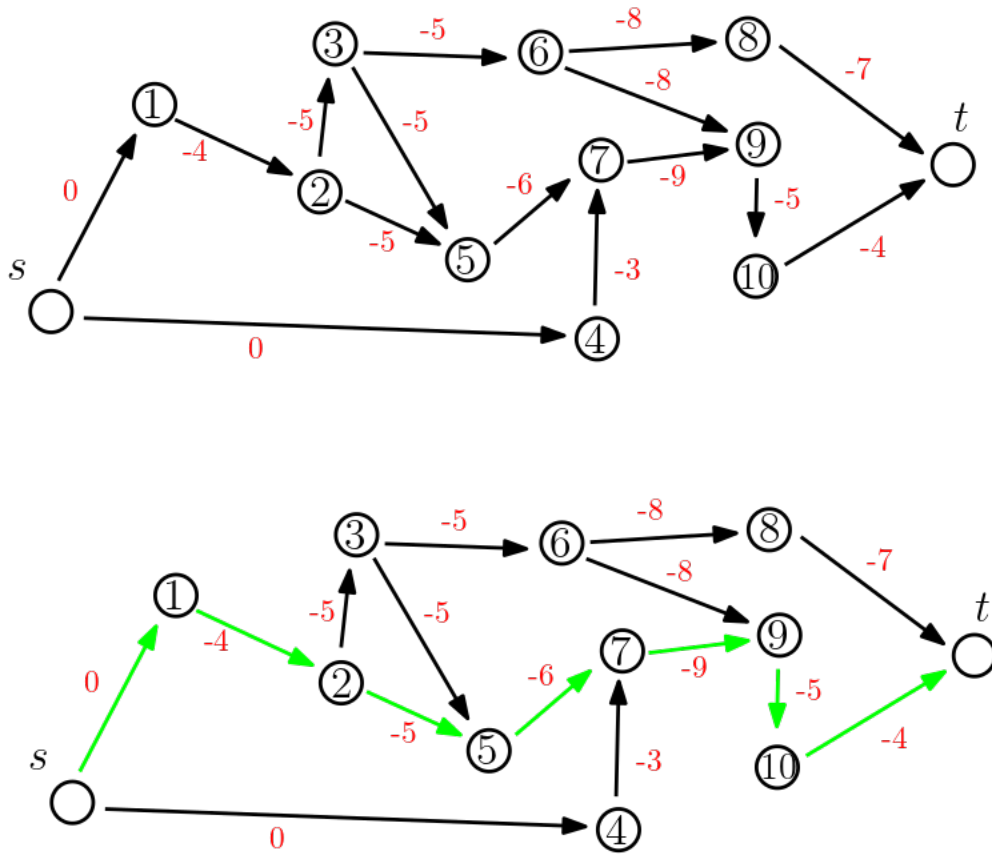


Figure 3: Top figure: The network obtained from Figure 2 by replacing each cost c_a with $-c_a$. To find the completion time of the project, we want to solve the shortest path problem in this network. Note that the network is acyclic. Bottom figure: In green, a Minimum Cost Flow in the instance obtained from the network on top by setting costs as on top, capacity 1 on each arc, demands $d_s = -1$, $d_t = 1$, and $d_i = 0$ for every other node. Thus, the project completion time is equal to the cost of the path in green with respect to the original cost vector c , which is 33.

all, since such a cycle would imply that there is a circular sequence of activities that are one the prerequisite of the other, a contradiction to the fact that the project management problem can be solved at all.

Summarizing, to solve the project management problem we follow the steps below:

1. Model the interdependencies among activities as a longest $s - t$ path problem in an acyclic network $N(V, A)$ with costs c .
2. Replace c with $-c$.
3. Solve the Shortest Path problem from s to t in $N(V, A)$ with costs $-c$ by reducing it to a Minimum Cost Flow problem, as in Theorem 4.
4. The cost of the path found (with respect to the original cost c) gives the project completion time.

In Figure 3 we illustrate this approach for the example above.

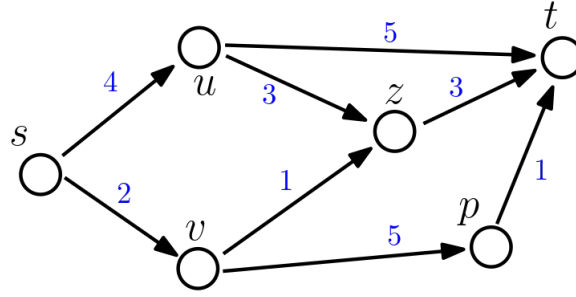


Figure 4: An instance of the Minimum $s - t$ cut / Maximum $s - t$ flow problem, with capacities denoted next to the corresponding edges.

3 Maximum $s - t$ Flow and Minimum $s - t$ Cut

In the *Minimum $s - t$ cut* problem, we are given a network $N(V, A)$ a capacity vector $u : A \rightarrow \mathbb{N}$, and nodes $s, t \in V$. We want to find an $s - t$ cut of minimum capacity, defined as follows. An $s - t$ cut is a set $S \subseteq V$ such that $s \in S, t \notin S$. We let $\delta^+(S)$ be the set of arcs $(i, j) \in A$ with $i \in S, j \in V \setminus S$, and the *capacity* of a cut S be the value

$$u(\delta^+(S)) := \sum_{i \in S, j \in V \setminus \{s\}} u(i, j).$$

For instance, in Figure 4, sets $\{s, u, v\}$ and $\{s, u, z\}$ are $s - t$ cuts, of capacity $5 + 3 + 1 + 5 = 14$ and $2 + 5 + 3 = 10$ respectively. Thus, in the minimum $s - t$ cut problem, we prefer the latter to the former. On the other hand, $\{s, u, t\}$ is not an $s - t$ cut, since it contains node t . Before we discuss how to solve the Minimum $s - t$ cut problem, let us see some applications.

3.1 Open-pit mining

In the Open-pit mining problem we are given a set of n blocks that can be excavated in a mine. Excavating block i leads to a profit p_i which can be positive (if block i contains precious stones) or negative (if block i contains only rocks). There are a set of precedence relations² between blocks, of the kind

Before we excavate block i , we need to excavate block j .

The goal is to find the set of blocks to excavate, so as to maximize the profit of the excavated blocks.

Consider the example in Figure 5. Each square is a block, and inside each block we denote the associated profit. An arc (i, j) describes a preference relation: before we excavate block i , we need to excavate block j . Thus, the solution, taking blocks with profit 1, 3, -5, -20, -4, 2, -1, 58, 10 is feasible and leads to profit 44, while the solution obtained from the one above without excavating the block with profit -20 is not feasible (since, for instance, it precedes the block with profit 2).

The Open-pit mining problem can be solved via minimum-cut as follows:

- Let B_+ = blocks with positive profit, B_- = blocks with negative profit.

²These relations model, for instance, two blocks that lie vertically one above the other, so it is physically impossible to reach the one below if we have not excavated the one above.

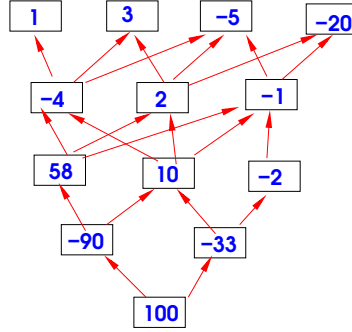


Figure 5: An instance of the Open-pit mining problem.

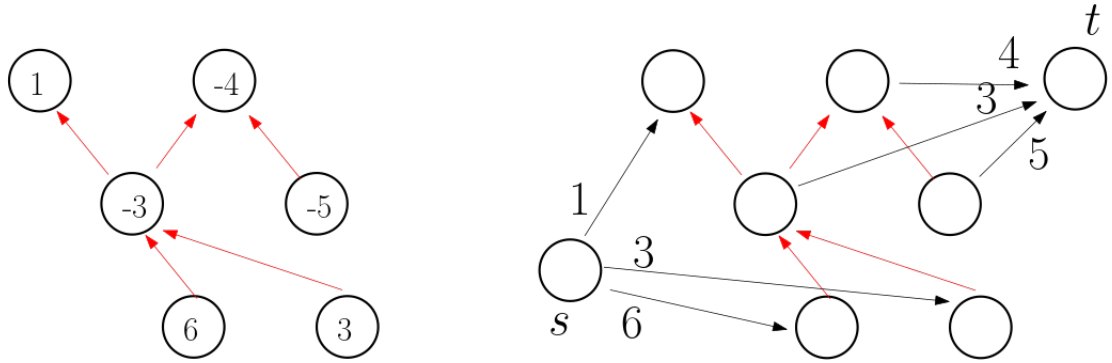


Figure 6: On the left: An instance of the Open-pit mining problem. On the right: the corresponding instance of Minimum $s - t$ cut, whose optimal solution takes all nodes except the one with profit -5 and t . Thus, the optimal solution to the open-pit problem is given by excavating all nodes except the one with profit -5 .

- Create a Network N as follows:
 - Nodes = blocks $\cup \{s, t\}$
 - For a block i preceding a block j , create an arc (j, i) with capacity $M \gg 0$;
 - For a block $i \in B_-$, create an arc (i, t) of capacity $-p_i$;
 - For block $i \in B_+$, create an arc (s, i) of capacity p_i ;
- Compute a Minimum $s - t$ cut S^* in the network;
- $S^* \setminus \{s\}$ is an optimal solution to the open-pit mining problem.

See Figure 6 for an example.

3.2 Image segmentation

In the Image Segmentation problem, we have a picture which is given by a collection of pixels. Each pixel is represented by a vector $x \in \mathbb{Z}_{\geq 0}^3$, where each component denotes the amount of Red, Green, and Blue color contained in the pixel (hence, we talk of a pixel as a point in the RGB space). In the image, there is an object O and a background



Figure 7: A possible input to the image segmentation problem. Source: https://commons.wikimedia.org/wiki/File:Intelligent_scissors_segmentation.png

B. We want an automatic way to isolate pixels in O . Image Segmentation techniques are used in various settings, including medical imaging, autonomous vehicles, the analysis of satellites images, etc.

Here is a possible approach to the problem. For pixels i and j represented by vectors x and y in the RGB space, define their distance to be

$$d(i, j) = |x_1 - y_1| + |x_2 - y_2| + |x_3 - y_3|.$$

Notice that the more different the pixels are, the larger the distance. We solve the Image Segmentation problem as follows:

- Create a Network where:
 - Each node is a pixel;
 - There is an arc (i, j) and an arc (j, i) of cost $2^{-d(i, j)}$ between adjacent pixels i, j ;
- Ask the user to select a pixel in O , name it s ;
- Ask the user to select a pixel in B , name it t ;
- Find a minimum $s - t$ cut S^* in the network;
- Nodes in S^* correspond to pixels of the object; iterate, if needed.

The rationale for the approach above is the following. We can expect that, at the boundary between the Object and the Background, there is a sudden change of color, as one can see in Figure 7. Thus, there will be adjacent pixels i, j (one in O , one in B) with $d(i, j)$ large, which corresponds to an arc with small cost. In an optimal solution to the Minimum $s - t$ Cut problem we will therefore select a set S of pixels that contain s and whose adjacent pixels not in S are “very different”. Thus, S is a good candidate to isolate the pixels of the object from the rest. Note that the algorithm may isolate O together with some other component that is also very different from the background, in which case we can repeat the process.

3.3 Solution of the Minimum $s - t$ cut problem via the Maximum $s - t$ flow

Recall that, in the Maximum $s - t$ flow problem, we are given a Network $N(V, A)$, source $s \in V$, sink $t \in V \setminus \{s\}$, and capacities $u : A \rightarrow \mathbb{N}$. The goal is to find, among the flows from s to t that respect capacity, one of *maximum*

value, where the value $val(x)$ of a flow is given by the *net flow* from s (that is, the flow leaving s minus the flow entering s) or equivalently by the net flow into t (that is, the flow entering t minus the flow leaving t).

Recall that we saw, in a previous class, that the maximum flow problem can be formulated, and solved, using Linear Programming. We start by observing that a theorem analogous to Theorem 1 holds for the Maximum $s - t$ flow problem.

Theorem 5. *If in an instance of the Maximum $s - t$ Flow problem vector u is integer, then there is integer optimal flow, which is also a bfs.*

Thus, once again, when we solve a Maximum Flow problem (with integer capacities) we will get an integer solution, a nice feature to have.

Let us now investigate the relationship between flows and cuts. First observe that the capacity of any $s - t$ cut S gives an upper bound to the value of every $s - t$ flow. Indeed, any unit of flow from s to t needs to “exit” set S and, to do so, it must traverse one of the arcs leaving S (and consume the corresponding capacity). Thus, the total flow that “exists” S is upper bounded by the capacity of the cut. We summarize this observation in the following.

Theorem 6 (Weak Duality between flows and cuts). *Consider an instance to the Maximum $s - t$ flow problem, and observe it is also an instance to the Minimum $s - t$ cut problem. Let x be an $s - t$ flow and S an $s - t$ cut. Then*

$$val(x) \leq u(\delta^+(S)).$$

Similarly to what we observed between dual problems in Linear Programming, at the optima the value of the $s - t$ flow and the capacity of the $s - t$ cut coincide.

Theorem 7 (Strong Duality between flows and cuts). *Consider an instance to the Maximum $s - t$ flow problem, and observe it is also an instance to the Minimum $s - t$ cut problem. Then x^* is a maximum $s - t$ flow and S^* is a minimum $s - t$ cut if and only if*

$$val(x^*) = u(\delta^+(S^*)).$$

Although we will not prove the results above, it is not surprising that one could show them using Linear Programming Duality³, associating to the primal Maximum $s - t$ flow LP a dual LP representing the Minimum $s - t$ Cut problem. Indeed, one can use such a connection to obtain an optimal solution to the Minimum $s - t$ Cut problem from the optimal dual of the Maximum $s - t$ Flow LP, as discussed in the next theorem.

Theorem 8. *Consider an instance of the Minimum $s - t$ Cut problem and the LP associated to the corresponding Maximum $s - t$ flow problem:*

$$\begin{aligned} \max \quad & \underbrace{\sum_{a=(u,t) \in A} x_a}_{\text{flow into } t} - \underbrace{\sum_{a=(t,u) \in A} x_a}_{\text{flow out of } t} \\ & \underbrace{\sum_{a=(u,v) \in A} x_a}_{\text{flow into } v} - \underbrace{\sum_{a=(v,u) \in A} x_a}_{\text{flow out of } v} = 0 \quad \text{for all } v \in V \setminus \{s, t\} \quad [y_v] \\ & 0 \leq x_a \leq u_a \quad \text{for all } a \in A \quad [\pi_a] \end{aligned}$$

Let $(\bar{y}, \bar{\pi})$ be an optimal solution to the dual of the LP above and define

$$S^* := \{s\} \cup \{v \in V \setminus \{s, t\} : \bar{y}(v) \geq 0\}.$$

³There are also other proofs, that do not directly use LP duality.

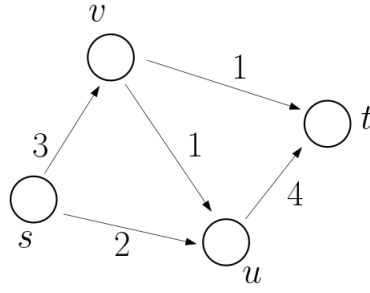


Figure 8: An instance of the Maximum $s - t$ flow and Minimum $s - t$ cut problems.

Then S^* is an $s - t$ cut of minimum capacity, i.e.,

$$\text{val}(x^*) = u(\delta^+(S^*)).$$

Let's see next an example of the application of the theorem above.

Example. Consider the instance of Maximum $s - t$ flow / Minimum $s - t$ cut from Figure 8. We can write the flow LP as follows:

$$\begin{array}{rcllcl}
 \max & & x_{v,t} & +x_{u,t} & & \\
 & x_{s,v} & -x_{v,u} & -x_{v,t} & = & 0 \quad [y_v] \\
 & & x_{s,u} & +x_{v,u} & -x_{u,t} & = 0 \quad [y_u] \\
 & x_{s,v} & & & & \leq 3 \quad [\pi_{s,v}] \\
 & & x_{s,u} & & & \leq 2 \quad [\pi_{s,u}] \\
 & & & x_{v,u} & & \leq 1 \quad [\pi_{v,u}] \\
 & & & & x_{v,t} & \leq 1 \quad [\pi_{v,t}] \\
 & & & & & x_{u,t} \leq 4 \quad [\pi_{u,t}] \\
 & & & & & x \geq 0
 \end{array}$$

where next to each constraint, we denote the corresponding dual variables.

Its dual is obtained following the usual conversion table:

$$\begin{array}{rcllcl}
 \min & & 3\pi_{s,v} & +2\pi_{s,u} & +\pi_{v,u} & +\pi_{v,t} & +4\pi_{u,t} & & \\
 & y_v & & +\pi_{s,v} & & & & \geq & 0 \quad [x_{s,v}] \\
 & & y_u & & +\pi_{s,u} & & & \geq & 0 \quad [x_{s,u}] \\
 & -y_v & +y_u & & & +\pi_{v,u} & & \geq & 0 \quad [x_{v,u}] \\
 & -y_v & & & & & +\pi_{v,t} & \geq & 1 \quad [x_{v,t}] \\
 & & -y_u & & & & +\pi_{u,t} & \geq & 1 \quad [x_{u,t}] \\
 & & & & y & & & \geq & \text{free} \\
 & & & & \pi & & & \geq & 0
 \end{array}$$

By e.g. using a solver, one can obtain the optimal primal solution x^* :

$$x_{s,v}^* = 2, x_{s,u}^* = 2, x_{v,u}^* = 1, x_{u,t}^* = 4, x_{v,t}^* = 1, x_{u,t}^* = 3,$$

and the optimal dual solution (y^*, π^*) :

$$y_v^* = 0, y_u^* = -1, \pi_{s,v}^* = 0, \pi_{s,u}^* = 1, \pi_{v,u}^* = 1, \pi_{u,t}^* = 0, \pi_{v,t}^* = 1.$$

From Theorem 8, we conclude that a minimum $s - t$ cut is given by $S^* = \{s, v\}$.